

Relatório Técnico Completo do Projeto

1. Fonte dos Dados

Os dados são provenientes de arquivos CSV públicos do Campeonato Brasileiro de Futebol, incluindo:

- campeonato-brasileiro-cartoes.csv: Cartões (amarelos/vermelhos) por partida.
- campeonato-brasileiro-estatisticas-full.csv: Estatísticas detalhadas de mandantes e visitantes.
- campeonato-brasileiro-full.csv: Dados gerais das partidas (placar, datas, técnicos, etc.).
- campeonato-brasileiro-gols.csv: Registro de gols (jogador, minuto, tipo de gol).

Pré-processamento

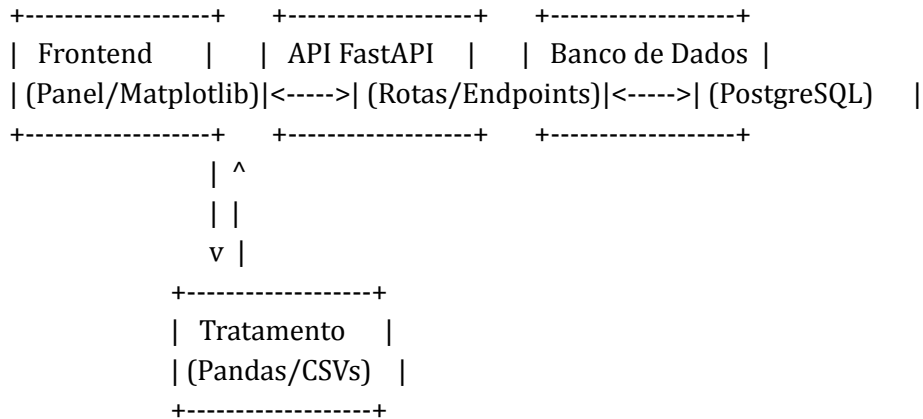
- Tratamento de dados em tratamento.ipynb para corrigir inconsistências (ex: preenchimento de posições de atletas, normalização de porcentagens).
- Separação de estatísticas em tabelas estatisticas_mandantes e estatisticas_visitantes.
- Criação de IDs sequenciais e relacionamentos via SQLAlchemy.

2. Tecnologias Utilizadas

Categoria	Tecnologias/Ferramentas
Backend	FastAPI, SQLAlchemy (ORM), Pydantic (validação), AsyncIO (assincronia)
Banco de Dados	PostgreSQL (container Docker)
Visualização	Panel (interface web), Matplotlib (gráficos), Tabulator (tabelas interativas)
Infraestrutura	Docker, Docker Compose (orquestração)
Processamento	Pandas (manipulação de CSVs), Jupyter Notebook (análise exploratória)
Versionamento	Git, GitHub

3. Arquitetura do Sistema

Diagrama de Componentes



Estrutura do Projeto

```
PROJETO-FINAL-PERSISTENCIA/
├── app/
│   ├── models/          # Modelos de dados (SQLAlchemy e DTOs)
│   ├── routes/          # Endpoints da API
│   ├── database_util.py  # Conexão assíncrona com PostgreSQL
│   └── main.py           # Inicialização da aplicação
├── data_mining/          # Datasets e scripts SQL
├── docker-compose.yml    # Configuração do PostgreSQL e app
├── Dockerfile            # Build da imagem da aplicação
├── requirements.txt      # Dependências Python
└── README.md            # Instruções de uso
```

4. Funcionalidades Principais

4.1. CRUD Básico

- ****Partidas****: POST /partidas → Cria uma nova partida.
- ****Gols****: GET /gols/jogador/{nome} → Lista todos os gols de um jogador.
- ****Cartões****: GET /cartoes/jogador/{nome} → Cartões recebidos por um jogador.

4.2. Consultas Avançadas

Endpoint	Descrição
----------	-----------

GET /partidas/vitorias	Compara vitórias entre dois clubes (ex: Flamengo vs. Palmeiras).
GET /gols/contagem-por-tipo-e-clube	Retorna gols agrupados por tipo e clube.
GET /cartoes/contagem-por-clube	Contagem de cartões amarelos/vermelhos por clube.
GET /partidas/estatisticas	Médias e totais de gols (mandante vs. visitante).

4.3. Visualização de Dados

- **Gráficos**: GET /partidas/grafico/gols → Gera um gráfico de linha com gols por data (PNG).

5. Instruções para Execução

5.1. Pré-requisitos

- Docker e Docker Compose instalados.
- Python 3.12 (opcional, apenas para execução local sem Docker).

5.2. Passos

1. **Clone o Repositório**:

git clone github.com/juandbpimentel/Projeto-Final-Persistencia

outras instruções estão no readme do projeto

2. **Suba os Containers**:

```
docker-compose up --build
```

5.3. Variáveis de Ambiente

Configure no docker-compose.yml:

environment:

POSTGRES_USER: postgres

POSTGRES_PASSWORD: postgres

POSTGRES_DB: postgres

DATABASE_URL: "postgresql+asyncpg://postgres:postgres@db:5432/postgres"

Créditos

Gabriel Correia fez os métodos e métodos complexos de partidas.

Juan fez os CRUDs.

Wilhelm fez a conexão com o Docker e a estruturação do projeto, juntamente com o Juan.

O CRUD de gols e cartões, e algumas consultas complexas nessas duas entidades foram feitas por Wilhelm.

Guilherme fez o tratamento de dados.

Iago testou algumas rotas com o Juan e, como seu notebook não estava rodando o Docker está fez **pair programming** com o Juan.